

Star Force Base 1198 — Project Readme

Overview

This project uses the following tech stack:

- Vite
- Typescript
- React Router v7 (all imports from react-router instead of react-router-dom)
- React 19 (for frontend components)
- Tailwind v4 (for styling)
- Shadcn UI (for UI components library)
- Lucide Icons (for icons)
- Convex (for backend & database)
- Convex Auth (for authentication)
- Framer Motion (for animations)
- Three.js (for 3d models)

All relevant files live in the 'src' directory.

Use bun for the package manager.

Admin Guide

Site owners: see ADMIN-GUIDE.md — a plain-language guide to where your data lives, how to administer the site (Operator Console + Convex dashboard), and how changes reach the live site. Technical deployment details live in DEPLOYMENT.md.

Setup

This project is set up already and running on a cloud environment, as well as a convex development in the sandbox.

Environment Variables

The project is set up with project specific CONVEX_DEPLOYMENT and VITE_CONVEX_URL environment variables on the client side.

The convex server has a separate set of environment variables that are accessible by the convex backend.

Currently, these variables include auth-specific keys: JWKS, JWT_PRIVATE_KEY, and SITE_URL.

Using Authentication (Important!)

You must follow these conventions when using authentication.

Auth is already set up.

All convex authentication functions are already set up. The auth currently uses email OTP and anonymous users, but can support more.

The email OTP configuration is defined in src/convex/auth/emailOtp.ts. DO NOT MODIFY THIS FILE.

Also, DO NOT MODIFY THESE AUTH FILES: src/convex/auth.config.ts and src/convex/auth.ts.

Using Convex Auth on the backend

On the src/convex/users.ts file, you can use the getCurrentUser function to get the current user's data.

Using Convex Auth on the frontend

The /auth page is already set up to use auth. Navigate to /auth for all log in / sign up sequences.

You MUST use this hook to get user data. Never do this yourself without the hook:

```
import { useAuth } from "@/hooks/use-auth";

const { isLoading, isAuthenticated, user, signIn, signOut } = useAuth();
```

Protected Routes

When protecting a page, use the auth hooks to check for authentication and redirect to /auth.

Auth Page

The auth page is defined in src/pages/Auth.tsx. Redirect authenticated pages and sign in / sign up to /auth.

Authorization

You can perform authorization checks on the frontend and backend.

On the frontend, you can use the useAuth hook to get the current user's data and authentication state.

You should also be protecting queries, mutations, and actions at the base level, checking for authorization securely.

Adding a redirect after auth

In src/main.tsx, you must add a redirect after auth URL to redirect to the correct dashboard/profile/page that should be created after authentication.

Frontend Conventions

You will be using the Vite frontend with React 19, Tailwind v4, and Shadcn UI.

Generally, pages should be in the src/pages folder, and components should be in the src/components folder.

Shadcn primitives are located in the src/components/ui folder and should be used by default.

Page routing

Your page component should go under the src/pages folder.

When adding a page, update the react router configuration in src/main.tsx to include the new route you just added.

Shad CN conventions

Follow these conventions when using Shad CN components, which you should use by default.

- Remember to use "cursor-pointer" to make the element clickable
- For title text, use the "tracking-tight font-bold" class to make the text more readable
- Always make apps MOBILE RESPONSIVE. This is important
- AVOID NESTED CARDS. Try and not to nest cards, borders, components, etc. Nested cards add clutter and make the app look messy.
- AVOID SHADOWS. Avoid adding any shadows to components. Stick with a thin border without the shadow.
- Avoid skeletons; instead, use the loader2 component to show a spinning loading state when loading data.

Landing Pages

You must always create good-looking designer-level styles to your application.

- Make it well animated and fit a certain "theme", ie neo brutalist, retro, neumorphism, glass morphism, etc

Use known images and emojis from online.

If the user is logged in already, show the get started button to say "Dashboard" or "Profile" instead to take them there.

Responsiveness and formatting

Make sure pages are wrapped in a container to prevent the width stretching out on wide screens. Always make sure they are centered aligned and not off-center.

Always make sure that your designs are mobile responsive. Verify the formatting to ensure it has correct max and min widths as well as mobile responsiveness.

- Always create sidebars for protected dashboard pages and navigate between pages
- Always create navbars for landing pages
- On these bars, the created logo should be clickable and redirect to the index page

Animating with Framer Motion

You must add animations to components using Framer Motion. It is already installed and configured in the project.

To use it, import the motion component from framer-motion and use it to wrap the component you want to animate.

Other Items to animate

- Fade in and Fade Out
- Slide in and Slide Out animations
- Rendering animations
- Button clicks and UI elements

Animate for all components, including on landing page and app pages.

Three JS Graphics

Your app comes with three js by default. You can use it to create 3D graphics for landing pages, games, etc.

Colors

You can override colors in: `src/index.css`

This uses the oklch color format for tailwind v4.

Always use these color variable names.

Make sure all ui components are set up to be mobile responsive and compatible with both light and dark mode.

Set theme using dark or light variables at the parent className.

Styling and Theming

When changing the theme, always change the underlying theme of the shad cn components app-wide under `src/components/ui` and the colors in the `index.css` file.

Avoid hardcoding in colors unless necessary for a use case, and properly implement themes through the underlying shad cn ui components.

When styling, ensure buttons and clickable items have pointer-click on them (don't by default).

Always follow a set theme style and ensure it is tuned to the user's liking.

Toasts

You should always use toasts to display results to the user, such as confirmations, results, errors, etc.

Use the shad cn Sonner component as the toaster. For example:

```
import { toast } from "sonner"

import { Button } from "@/components/ui/button"
export function SonnerDemo() {
  return (
    <Button
      variant="outline"
      onClick={() =>
        toast("Event has been created", {
          description: "Sunday, December 03, 2023 at 9:00 AM",
          action: {
            label: "Undo",
            onClick: () => console.log("Undo"),
          },
        })
      }
    >
    Show Toast
  </Button>
  )
}
```

Remember to import { toast } from "sonner". Usage: toast("Event has been created.")

Dialogs

Always ensure your larger dialogs have a scroll in its content to ensure that its content fits the screen size. Make sure that the content is not cut off from the screen.

Ideally, instead of using a new page, use a Dialog instead.

Using the Convex backend

You will be implementing the convex backend. Follow your knowledge of convex and the documentation to implement the backend.

The Convex Schema

You must correctly follow the convex schema implementation.

The schema is defined in src/convex/schema.ts.

Do not include the _id and _creationTime fields in your queries (it is included by default for each table). Do not index _creationTime as it is indexed for you. Never have duplicate indexes.

Convex Actions: Using CRUD operations

When running anything that involves external connections, you must use a convex action with "use node" at the top of the file.

You cannot have queries or mutations in the same file as a "use node" action file. Thus, you must use pre-built queries and mutations in other files.

You can also use the pre-installed internal crud functions for the database:

```
// in convex/users.ts
import { crud } from "convex-helpers/server/crud";
import schema from "../schema.ts";

export const { create, read, update, destroy } = crud(schema, "users");

// in some file, in an action:
const user = await ctx.runQuery(internal.users.read, { id: userId });

await ctx.runMutation(internal.users.update, {
  id: userId,
  patch: {
    status: "inactive",
  },
});
```

Common Convex Mistakes To Avoid

When using convex, make sure:

- Document IDs are referenced as `_id` field, not `id`.
- Document ID types are referenced as `Id<"TableName">`, not `string`.
- Document object types are referenced as `Doc<"TableName">`.
- Keep `schemaValidation` to `false` in the schema file.
- You must correctly type your code so that it passes the type checker.
- You must handle `null` / `undefined` cases of your convex queries for both frontend and backend, or else it will throw an error that your data could be `null` or `undefined`.
- Always use the `@/folder` path, with `@/convex/folder/file.ts` syntax for importing convex files.
- This includes importing generated files like `@/convex/_generated/server`, `@/convex/_generated/api`
- Remember to import functions like `useQuery`, `useMutation`, `useAction`, etc. from `convex/react`
- NEVER have return type validators.